

1 Optimization / gradient descent (Q&A)

1.1 Q: What does gradient descent update in these models?

A: Gradient-based optimizers update every learnable tensor returned by `model.parameters()` (embeddings, linear layers, MLP weights, and mixture-head weights), because those parameters determine the predicted distribution/score and therefore the loss.

1.2 Q: What objective is being minimized for EGMN in training?

A: The training script minimizes $\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}}$, i.e. mixture negative log-likelihood plus an entropy term and an L1 reconstruction term weighted by α, β .

1.3 Q: What does one optimizer step look like in code?

A:

```
optimizer = torch.optim.Adagrad(model.parameters(), lr=args.lr,
                                weight_decay=args.weight_decay)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

1.4 Q: How do gradients flow to EGMN's mixture outputs?

A: The outputs $\pi, \lambda, \mu, \sigma$ are not free parameters but functions of the network weights, so back-propagation differentiates the loss through these outputs into the head layers and shared trunk that produced them.

1.5 Q: Which things are not updated by gradient descent?

A: Inputs (feature/label tensors), preprocessing artifacts (e.g. bucket boundaries and quantile tables), and computations inside `torch.no_grad()` are not updated because they are not learnable parameters in the optimizer.